Trabalho Prático de Programação Modular – Parte 2

Este trabalho é um continuação do TP1. Todos os problemas do TP1 devem ser corrigidos para o TP2, exceto na camada de visão, que será substituída por completo. Todas as mudanças estipuladas no TP2 em relação ao TP1 devem ser realizadas.

Pacote modelo

Entidade deve ter a igualdade comparada com base no atributo id. Para isso, sobrescreva o método **equals** de Object e o método **compareTo** de Comparable em Java ou os métodos vinculados aos operadores relacionais de Python para o atributo id.

Pacote persistência

Deve ser criada uma classe de exceção **PersistenceException** que apresenta uma mensagem indicando a operação (salvar, atualizar, apagar, carregar um ou todos), o problema que ocorreu e o valor (objeto de uma subclasse de Entidade ou id) que causou a exceção.

A classe **EntidadeDAO** passar a ser genérica com um parâmetro de tipo E que estende Entidade. O atributo que é um conjunto (Set) passar a ser do tipo E. Além disso, os métodos de persistência também devem ser alterados da seguinte forma:

- **salvar**, que recebe um objeto do tipo E e o adiciona no conjunto. Não tem retorno, mas lança uma PersistenceException se o objeto já existir no conjunto ou outro problema ocorrer;
- **atualizar**, que recebe um objeto do tipo E que substitui o elemento existente no conjunto que possui o mesmo id. Não tem retorno, mas lança uma PersistenceException se não existir um objeto com mesmo id no conjunto;
- **apagar**, que recebe um id e remove e retorna o elemento existente no conjunto que possui o mesmo id. Lança uma PersistenceException se não existir um objeto o id indicado no conjunto;
- **buscar**, que recebe um id retorna o objeto no conjunto que possui o id indicado, lança uma PersistenceException se não existir um objeto com mesmo id no conjunto;
- **carregar**, método que retorna um array (array mesmo, não lista nem o próprio conjunto) com todos os objetos armazenados no conjunto ordenados por id. Lança uma PersistenceException se o conjunto estiver vazio.
- **persistir**, que persiste o conteúdo do conjunto em um arquivo.
- **recuperar**, que carrega o conjunto com as entidades contidas em um arquivo.

Crie uma classe que garante que exista apenas uma única EntidadeDAO por entidade em todo o sistema e, dessa forma, impedindo ambiguidades na persistência de cada entidade. As classes dos demais pacotes realizam as operações de persistência a partir dessa classe.

Pacote visao

A interface com o usuário por meio de linha de comando do sistema deve ser substituída por uma interface gráfica. Deve haver uma janela principal e uma ou mais janelas/painéis para cada entidade. A partir da janela principal deve ser possível chegar nas demais janelas ou painéis.

O painel/janela de cada entidade deve possuir uma tabela que permite visualizar todos os registros existentes no conjunto daquela entidade. Deve ser possível ordenar a tabela a partir de qualquer coluna. Também deve possuir um painel ou janela para cadastrar, apagar, editar e buscar (carregar).

O programa deve realizar tratamentos de exceção para dados digitados incorretamente (por exemplo, letras onde devem haver apenas números e datas em formatos inválidos).

A persistência ou recuperação do conjunto de cada entidade no arquivo pode ser realizada a cada operação de persistência realizada ou quando a janela relacionada com essa entidade é aberta ou fechada, respectivamente.

Pacote testes

Adicione no programa esse novo subpacote e crie nele uma classe para cada EntidadeDAO contendo os seguintes casos de testes para cada operação:

- **salvar**: um caso de teste que insere uma entidade com id ainda não existente, conferindo se ela realmente foi inserida; um caso de teste que insere uma entidade com id já existente, conferindo se ela realmente não foi inserida e se a exceção ocorreu;
- **atualizar**: um caso de teste que altera uma entidade com id não existente, conferindo se nada foi feito e se a exceção ocorreu; um caso de teste que altera uma entidade com id existente, conferindo se ela realmente foi alterada;
- **apagar**: um caso de teste que remove uma entidade com id não existente, conferindo se nada foi feito e exceção ocorreu; um caso de teste que remove uma entidade com id já existente, conferindo se ela realmente foi apagada e se a saída do método foi a esperada para esse caso;
- **buscar**: um caso de teste que busca uma entidade com id não existente, conferindo se a exceção ocorreu; um caso de teste que busca uma entidade com id existente, conferindo se a saída do método foi a esperada para esse caso;